



Open Finance Data Security Standard (OFDSS)

Version 1.2
Updated 01/29/2022

INTRODUCTION

We have developed this Open Finance Data Security Standard (OFDSS) in an effort to enhance data security and establish a baseline set of security expectations for the participants in the Open Finance ecosystem. This data security standard establishes security requirements across 13 control domains that address common data security risks. These requirements are contextualized with implementation guides, along with audit steps for ensuring compliance. These requirements are not intended to exhaustively address all data security risks that may be material to any particular organization. However, these requirements will address security risks that are commonly encountered by emerging financial technology companies when processing or storing sensitive information.

INTENDED AUDIENCE

The intended audience for this standard are the technology-focused start-up and growth-stage organizations that are in the process of building and scaling their information security capabilities. If your organization already has a mature information security program, and the capacity to provide reasonable assurance about the effectiveness of that program, then your organization is likely to be already meeting the requirements captured in this standard. To determine the applicability of this standard to your relationship with Plaid, please refer to the terms of the Plaid master service agreement.

THIS STANDARD VS. OTHER FRAMEWORKS

The control domains captured in this standard will align with common criteria found in other security frameworks such as SSAE18 TSC for Security and NIST CSF. However, this standard will not align 1-to1 with all the control domains found in such frameworks as some control domains (e.g. physical and environmental controls) are intentionally omitted from this standard in an effort to optimize its usefulness for cloud-native, technology-focused start-up and growth-stage companies.

HOW TO USE THIS STANDARD

This standard consists of the four key elements as described below.

1. **Security Requirements:** This section establishes security requirements across the 13 domains.
2. **Requirement Context:** This section provides context and clarification as to the objective of the requirement, and what risks the requirement is designed to mitigate against.
3. **Implementation Guides:** This section provides implementation guidance and expected outcomes for each of the 79 security requirements. Note that for some of the requirements, additional implementation guidance is provided in a separate section of this document.
4. **Audit steps:** The audit steps captured in the current version of this document are intended to enable an auditor to obtain a minimum level of assurance that an organization has appropriately designed controls in place to address the risk domains captured in this standard. These audit steps are not intended to test the operational effectiveness of the individual controls.

HOW TO PROVIDE FEEDBACK

The usefulness and quality of this document will depend on the relevance of the topics covered. Therefore, please help us keep this document relevant by emailing your feedback to info@ofdss.org

CONTRIBUTORS

Shano Fonseka Head of Risk | Plaid
Kyle Barry Head of Security | Plaid
Peter David Security Engineering Lead | Plaid
Nitin Chauhan Security Engineering | Plaid
Jonathan Smith Security Engineering | Plaid
Jeet Damania Security Engineering | Plaid
Kenneth Moras Security Compliance Lead | Plaid
Patrick Albert Head of Security | Truework
Robbie Ostrow Platform Engineering | Vanta

FOUNDING SUPPORTERS

Flinks, MX, Plaid & Truework

SECURITY COMPLIANCE PARTNERS

Drata, Laika, Anecdotes, Secureframe & Vanta

The following table identifies the individual security requirements, along with context, implementation guides, and audit steps for each requirement.

NIST CSF Mapping: ID.AM-6, ID.BE-1, ID.BE-2, ID.BE-3, ID.GV-1, ID.GV-2, ID.GV-3, ID.GV-4, ID.RM-1, ID.RM-2, ID.RM-3, ID.SC-1, PR.DS-4, PR.IP-7, PR.IP-8, PR.IP-11

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Define Objectives: 1. Identify the security risks that are material to your business, applications, and environments. 2. Define how you mitigate the security risks that are material to your business, application, and environments. Establish Accountability: 3. Identify the team or individual that has overall accountability for identifying and mitigating security risks that are material to your business, applications, and environments. 4. Identify the teams or individuals responsible for each of the control domains that mitigate the risks that you have identified. For example, owners of controls in the following control domains: a. Asset Management b. Access Controls c. Change Controls d. Cryptography e. Data Minimization f. Incident Management g. Awareness and Training h. Auditing and Alerting i. Network Security j. Vendor Management	Defining objectives and establishing accountability enables appropriate resource allocation for ensuring information security. Defining Objectives: This enables an organization to identify the security risks that are relevant to them, and appropriately set the scope for mitigation activities (controls). Defining objectives is important to ensure alignment between the security risks that an organization faces with their mitigation activities. Establishing Accountability: This identifies individuals and teams who are responsible for ensuring information security, and control owners who are responsible for the operational effectiveness of security controls that they own.	<u>Defining objectives and accountability</u> – Information Security objectives and accountability for meeting those objectives are best captured in a formal information security policy. This policy should authorize action, establish objectives, and identify accountability for identifying and mitigating information security risks for your organization. Expected outcome: Your organization has a formal Information Security Policy that (1) defines objectives, (2) establishes accountability, (3) establishes scope, (4) is approved by individuals with management responsibilities, and (5) is kept up to date.	1. Request and review a current version of the organization's Information Security Policy. 2. Ensure the policy is up-to-date, approved by individuals that have sufficient authority, establishes expectations and scope, and identifies accountability. 3. Ensure that the policy minimally captures universally common security control domains.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Maintain visibility: - Establish a defined method of listing and identifying all assets and their criticality and sensitivity to the business - Assign responsibility for usage and security of each asset - Maintain clear documentation of network and data flow architectures - Maintain an inventory third party software and libraries in use - Define what is considered a healthy (and therefore trustable) state for each asset Prevent Dependency Confusion Attacks: - Use a centralized dependency / package manager to reduce the risk of dependency confusion attacks. - Ensure all internal dependencies are downloaded from internal registries for all production environments, including development and CI environments. - Whenever possible, verify the checksum of packages before installing.	A managed asset is one that an organization has visibility and control over, that is patched against vulnerabilities, and is protected against malicious code. These assets can be network endpoints, physical or virtual server instances, back-end services, source code repositories, and business applications. Visibility: An asset that you do not see cannot be managed. Therefore, continuous visibility into all critical assets is a prerequisite for securing them. Dependency Confusion Attacks: A misconfigured dependency manager (like Artifactory) can allow internal dependencies to be downloaded from public registries, leading to potentially malicious packages being installed on employee machines, containers, and servers. Ensuring that internal packages are only downloaded from private registries can protect your organization from falling victim to these types of attacks. Vulnerable third-party dependencies: In addition to in-house code, a significant portion of modern applications rely heavily on 3rd party code in the form of dependencies and libraries which often contain vulnerabilities. These vulnerable dependencies oftentimes become a silent killer of otherwise secure products. SCA	<u>Visibility</u> – Enterprise-grade network gateways, MDM tools, vulnerability scanners, and cloud service management consoles have the ability to discover and provide continuous visibility into your assets on the fly. Depending on the nature of components that make up your technology stack, it may be necessary to rely on a combination of these tools for maintaining high visibility (e.g. network gateway console for visibility into network endpoints connected to your corporate network, and cloud service management console for visibility into your cloud resources). The tools selected must have a mechanism for defining and maintaining the classification of the asset (e.g. production vs. development, or critical vs. non-critical). <u>Dependency Confusion Attacks</u> – Enforce the use of a central dependency manager with marked scopes for internal and 3rd party dependencies. All developers should be able to connect to the public registry through this central dependency manager. Direct access to public registries should be blocked on the CI/CD servers and on developer machines when possible. Define a process to publish internal dependencies and mandate a security review on them. Register all internal dependencies in respective public registries with the highest version number possible (e.g. 99999.99). Any time a new dependency is published internally, it should be registered before it is used in production assets. <u>Vulnerable third-party dependencies</u> – All business critical assets should have 3rd party dependency scanning (SCA) enabled, preferably integrated within the CI/CD environment, and regularly scanning for and identifying vulnerable dependencies (direct and transitive). Upgrade, remove, or replace any vulnerable	- Request a copy of the organization's assets inventory, and network or data flow diagram. - Ensure the assets inventory has a complete population that is consistent with known attributes of the company's technology stack (e.g. company uses cloud services, therefore, the inventory must capture cloud resources), and classifications for the identified assets (e.g. production vs. development). - Ensure that the organization has visibility into the usage of third-party dependencies in production code through the usage of centralized dependency or package managers. - Request a description of, or a document that describes, the company's strategy for mitigating against vulnerabilities and malicious code. - Ensure that the strategy for mitigating against vulnerabilities relies, minimally, on vulnerability scanners, has defined cadence and scope of scans,

Detect vulnerable third-party dependencies:	tools were created with the purpose of identifying such vulnerable libraries. Modern SCA tools have even evolved to the point of determining if a vulnerable piece of code inside the library is reachable and thus exploitable or if an active exploit is available for the vulnerability. Top tier offerings will go so far as to recommend safer versions of vulnerable libraries. Powerful tools like this help in understanding the criticality of vulnerabilities and greatly aid in triage prioritization.	libraries based on the vulnerability SLAs defined for your organization. Integrate SCA scanning on developer machines (e.g. in the IDE itself) and ensure that scans are run before code is committed.	has defined methods for determining materiality of vulnerabilities, and SLAs for deploying patches based on their materiality (vulnerability scans must happen at least once a month).
13. Deploy an SCA (Software Composition Analysis) solution to assist in creating a comprehensive dependency “bill-of-materials” and detecting vulnerable library versions. 14. Ensure that all library modules go through regular SCA scans.	First-party code vulnerabilities: Static analysis tools (SAST) can detect vulnerabilities very early on in the product development lifecycle. With the right well crafted rules, these SAST tools can output only actionable results, empowering developers to fix potential vulnerabilities as they write their code. This left-shifted approach can provide an organization maximum return on their investment by avoiding the future costs of fixing vulnerabilities, which can run up 35x–50x once vulnerable code is made live.	<u>First-party code vulnerabilities</u> – Document the process for running SAST, DAST, and IAST scans: • Identify a static analysis tool which works for your organization’s tech stack. • Evaluate rules to reduce false positive rates and integrate the SAST tool into the build process (CI/CD). • Identify and set up a dynamic analysis tool which works for your organization’s tech stack and set periodic scans on public internet facing applications. • Identify an IAST tool which works for your organization’s tech stack and bundle the IAST sensor with your application for deployment in the testing region, making sure to execute all business flows for the IAST tool to capture exploitable paths. • For any new code changes, a static analysis tool should scan the delta and report on any vulnerabilities found. • A periodic static scan should be set up on the complete code base to run an end to end analysis. • A security reviewer should triage vulnerabilities to identify true positives, exploitability, severity, etc. • All identified vulnerabilities should be fixed based on the SLA defined by your organization.	6. Ensure that the strategy for mitigating against malicious code relies on endpoint security tools that are centrally managed and kept up to date (virus / malware signatures must be updated at least once every 24 hours).
Detect first-party code vulnerabilities:	Static analysis tools (SAST) can detect vulnerabilities very early on in the product development lifecycle. With the right well crafted rules, these SAST tools can output only actionable results, empowering developers to fix potential vulnerabilities as they write their code. This left-shifted approach can provide an organization maximum return on their investment by avoiding the future costs of fixing vulnerabilities, which can run up 35x–50x once vulnerable code is made live.		
Patch vulnerabilities:	When deployed within the CI pipeline, SAST scanning can act as an additional layer of defense by detecting vulnerabilities before a product is deployed or even blocking the builds (depending on the threshold for vulnerabilities defined) if necessary. Dynamic (DAST) and Interactive Analysis (IAST) are powerful tools which can detect vulnerabilities during runtime giving them the power to identify vulnerabilities that may have gone undetected by static code scans.	<u>Patching</u> – Performing recurring scans against all discovered production assets using a robust vulnerability scanner is a good way to detect vulnerabilities. A process must be established to review the output of these scans, determine their materiality to your technology stack, and deploy available patches based on that materiality. Note that the default severity ratings rendered by a vulnerability scanner for findings are in absolute terms and not in relation to your technology stack. For example, it might	
Protect against malicious code:	Patching: Maintaining current patch levels for critical assets ensures that they are free of vulnerabilities that could lead to their compromise.		
20. Utilize malicious code prevention tools at interaction points with the public internet (mutable internet			

facing servers, laptops /
workstations, mobile devices, etc.).

Malicious code:

Malicious code (viruses, malware, trojans, etc.) is one of the most common methods used to compromise systems and data, therefore, it's not possible to secure devices without protecting those devices against malicious code.

find a critical SQL injection vulnerability in a library you use in a stack that does not use SQL. Although this is a "critical" vulnerability in absolute terms, it's not a "critical" vulnerability in relation to an application stack that does not have SQL in any of its paths. Therefore, materiality of vulnerabilities must be assessed in the context of the underlying stack, and not in just absolute terms.

Malicious code – The most common and an effective way to protect network endpoints (workstations and mutable server instances) against malicious code is to deploy endpoint security agents on all managed network endpoints. Most enterprise-grade endpoint security tools will provide a mix of features such as signature-based protections against viruses and malware, and host intrusion detection and prevention all through the same tool.

Expected outcome: Your organization has the ability to (1) produce an accurate asset inventory on demand, (2) detect and patch vulnerabilities in first code and third-party dependencies, and (3) protect critical assets against malicious code.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Deliberately control access: 21. Limit access grants to minimum necessary for job role and service roles 22. Require authorization for access grants to sensitive assets and data. 23. Identify and revoke obsolete access (e.g. upon user termination and when access is no longer needed) Ensure segregation of duties (SoD) 24. Prevent individuals from granting access to themselves, reviewing their own access, and approving their own access Use strong authentication: 25. Utilize industry-accepted strong protocols for authenticating users (e.g. something you know and something you have) 26. Escalate strength of authentication based on asset criticality. Harden points of authentication: 27. Ensure rate-limiting mechanisms are in place at points of authentication.	Preventing unauthorized access to assets is a key security objective, therefore, access and authentication controls are a critical component of a robust information security strategy, and is a key control. Deliberate access controls: Access to an asset should be the result of a controlled and deliberate action taken by the organization. If individuals and services are able to obtain and maintain access to assets due to reasons unknown to the organization, then the security of those assets cannot be ensured. Segregation of Duties (SoD): The purpose of access controls is to prevent unauthorized access, if individuals are allowed to review, approve, and provision their own access to an environment, unauthorized access cannot be prevented. Strong authentication: Strong authentication layers enable an organization to enforce access controls, and usernames and passwords are considered ineffective factors of authentication on their own. Using multiple factors of authentication (e.g. one time tokens) protects an asset against unauthorized access in the event that	<u>Controlling access</u> – Deliberately controlling access requires establishing and enforcing an internal access controls policy that defines the requirements for (1) granting access to an asset, (2) reviewing access grants to an asset, and (3) revoking access to an asset. This policy must identify roles and responsibilities for each of these activities, and require that an individual is only able to have the least number of permissions they need to perform their job responsibilities (this is the concept of least-privileged role-based access controls). A cadence for performing reviews of existing access grants must also be established, and SLAs for event-driven access revokes (e.g. when an employee leaves the company or changes roles) must also be defined. <u>Segregation of duties</u> – Segregation of duties between the grantor (individual performing the access grant), grantee (individual receiving the access), and the approver (individual authorizing the access) must be maintained. <u>Strong authentication</u> – Authentication for all critical assets must be deployed to require something that the user knows (e.g. username and password) and something that the user has in their possession (e.g. a push notification to an authenticator app in the user's possession). 2FA layers that rely on static responses, such as question-based 2FA, should not be used as they do not rely on something that the user has in their possession, therefore, are considered an ineffective form of 2FA.	1. Request and review current versions of the organization's policies and related documents that are relevant to access and authentication. 2. Ensure that these documents are current, establish requirements for granting, reviewing, and revoking access, and identify roles and responsibilities for each of these activities (there must be segregation of duties between grantor, grantee, and approvers of access). 3. Ensure that there are defined cadence and SLAs for recurring access reviews, and event driven revocation of access. Access reviews must occur at least annually, and access must be revoked for terminated users within 48 hours. 4. Ensure multiple factors of authentications are used when authenticating users to critical assets. Critical assets should require a strong form of MFA to authenticate users. Knowledge-based MFA (e.g. question and answer) should not be used as a MFA layer for critical assets. The preferred order of MFA is as follows, from strongest to weakest:

28. Ensure access defaults to DENY, and DENY in case of failure of the authentication mechanisms.	credentials (username and password) are compromised. Hardened points of authentication: Points of authentication are the critical paths of entry into an environment. Your organization should expect unauthorized users to attempt to breach these points of entry on a continuous basis, therefore, hardening these points of entry are key to preventing unauthorized access.	<u>Hardening points of authentication</u> – Authentication behavior of legitimate users tends to be predictable under most circumstances. The number of authentication attempts that a user is allowed should be defined, and requests that breach this threshold should be rate limited at points of authentication. Expected outcome: Your organization has (1) a deliberate strategy for controlling access to critical assets, (2) has deployed strong factors of authentication to all critical assets, and (3) hardened all points of authentication to critical assets.	a. Security Key (such as Yubikey) b. Push Notification to Device (such as Duo Push or Google Device Prompt or similar) c. Time-based One-Time Password (Google Authenticator App or similar with rotating code) d. SMS-based push OTP e. Email-based push OTP f. Question-based MFA 5. Ensure points of authentication use rate-limits to prevent successful brute force authentication attempts by unauthorized users.
---	--	---	---

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Prevent unauthorized changes: 29. Establish and maintain a clear process for requesting, developing, authorizing, deploying, and tracking changes 30. Ensure change control process applies to sensitive configuration changes 31. Ensure changes are evaluated for security risk, and adjust mitigating controls in response to change induced security risks Prevent untested changes: 32. Define testing requirements based on sensitivity of assets being changed 33. Standardize process for testing to ensure consistency across changes and accuracy of test results 34. Test for prevention of negative behavior as well as for the expected results from normal behavior of the change 35. Maintain testing library to protect against changes	Uncontrolled changes to assets can introduce unauthorized access and vulnerabilities. Therefore, controlling changes to critical assets is essential for information security (and high availability). Unauthorized changes: Changes to assets alter the way they behave and function, and if changes do not require review and authorization (approval), then such changes can be used to alter those assets in ways that compromises them. Testing: Testing of changes prior to their deployment to production enables you to detect unexpected behavior induced by the change. Deploying an untested change directly to production exposes that production asset to the risk of change induced security incidents. Version control: The ability to roll-back the behavior of an asset to a former stable / secure state is enabled by maintaining a robust history of changes to that asset (version control). If version control is missing from the build and release process, it can amplify the impact of change induced incidents. Segregation of Duties (SoD):	<u>Preventing unauthorized changes</u> – The development and deployment of code changes for production assets must be behind a build and release process that is gated by review and approval steps. These gating mechanisms must be enforced using logical controls (e.g. Branch protections functionality on GitHub) to ensure that they cannot be bypassed or sidestepped. <u>Testing</u> – Code changes to production assets must be tested using a consistent process before they can be deployed to production. This testing process should be standardized, and should be deployed as a logical gating mechanism for code changes. In other words, it should not be possible to deploy a change to a critical production asset if the tests for that change have not passed successfully. The common types of tests to integrate to a build and release process are: • Unit tests: This is a test of an individual piece of code or function that is being changed. The purpose of these tests are to ensure that the change is behaving as expected in isolation without any dependencies. For example, this test is comparable to individually tasting the ingredients you expect to use to assemble a sandwich. Each of these ingredients is a “unit” that can be “taste tested” before you assemble them into a sandwich. • Integration tests: This is a test of how the change functions with other dependent components. The purpose of these tests is to ensure that the components that were changed are behaving as expected when deployed with dependencies. For example, this test is comparable to assembling all the ingredients that would make up a	1. Request and review current versions of the organization's policies and related documents that are relevant to their build and release process or change controls process. 2. Ensure that these documents are current, establish requirements for approving, testing, version control, and segregation of duties. 3. Verify that changes, reviews, approvals, and evidence of testing are tracked using a ticketing system or an Source Code Management (SCM) tool. 4. Verify that approval, testing, version control, segregation of duties, and security code reviews is logically enforced using technical controls. 5. Verify that the organization has a defined strategy for standardizing configuration management and minimizing configuration drift when launching new services and infrastructure.

causing regressions to existing functionality and controls	Proper SoD eliminates conflicts of interest from your build and release process, and adds additional sets of eyes to any particular change or code commit. This helps ensure high quality changes, and reduces the likelihood of intentional or accidental change induced security incidents.	sandwich, and then taking a test bite of the assembled sandwich to confirm that it is indeed a sandwich.
Maintain version control:		Version Control – Production asset code repositories should live inside, and be managed through, a Source Code Management (SCM) tool. All modern SCM tools (such as Git) will have version control functionality to maintain a history of changes to code repositories managed by the SCM.
36. Maintain the ability to roll back changes quickly		
37. Ensure changes are tracked and searchable for future investigations	Automation: Manual processes increase the likelihood of user error, inconsistencies, and change induced security incidents. High quality automation in a build and release process will work to reduce the likelihood of these errors and help reduce the likelihood of change induced security incidents.	
38. Limit individual changes to small, related chunks of modifications to limit the risk of change induced incidents		Segregation of duties – Segregation of duties between the developer (change owner) and the approver (individual reviewing and approving the change) must be maintained. It should not be possible for the change owner and the approver to be the same individual for any particular code change, and this segregation of duties must be enforced logically using technical controls (e.g. Required Review functionality for Pull Requests on GitHub).
39. Establish clear release notes for substantive changes	Security Reviews of Code Changes: Security review of code changes, especially on business critical products, can help identify business flow exploitation and abuse, deviation from secure coding guidelines, and vulnerabilities that could compromise an application or product.	
Ensure segregation of duties (SoD):		Automation – Automated and standardized processes for testing code changes (continuous integration), building and configuring infrastructure (infrastructure-as-code), and deploying changes to production (continuous delivery) should be used instead of relying on manual processes.
40. Prevent individuals from approving their own changes		
Use automation:		Security Reviews of Code Changes – Define a code review checklist, which is to be frequently refreshed based on newly discovered patterns and needs. Code changes on business critical assets should have a security reviewer along with other peer reviewers whose approval is required for changes to be merged into the final release branch. Vulnerabilities identified during the review should be categorized based on business criticality of the asset, whether it will be hosted publicly, the severity of the vulnerability, etc., and triaged and remediated accordingly. Where manual review is not scalable, consider pairing manual reviews with automated static analysis.
41. Utilize automation tools for building, testing, and deploying code to production		
42. Standardize configuration management - such as infrastructure-as-code (IaC) - for provisioning and making changes to infrastructure		
Ensure security reviews of code changes:		
43. Identify critical components and make manual security		

review by a security engineer a mandatory step before code can be merged for production deployments.

44. Ensure a security focused review is done for all code changes prior to merging. The review can be performed by developers provided they follow company defined security standards.

Expected outcome: Your organization has (1) a defined process for building and releasing code changes to critical assets, (2) this process forces the testing of changes before they're deployed to production, (3) this process logically enforces review and approval of the code changes, (4) this process logically enforces segregation of duties between change owners and approvers/reviewers, (5) significant stages of the build and release process are automated to reduce user error, and (6) security reviews are performed for changes to business critical assets.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Define Secure Coding and Architecture Standards: 45. Define a per-language secure coding standard. 46. Define an extensible Secure Architecture Standard. 47. Ensure that the standard created encompasses common vulnerability patterns such as the OWASP top 10 or CWE top 25 vulnerabilities. 48. The standard should be reviewed and updated at least yearly. Establish early Threat Modeling: 49. Prioritize threat modeling for new/major design changes as early in the development process as possible (e.g. Security Reviews during the design phase of SDLC). 50. Periodically assess application trust boundaries, components, and data flows to account for any updates since the last threat model was completed.	Secure Coding and Architecture: Without secure coding and architecture standards in place, development and testing teams can be unaware of important security concepts. These teams implement what they know, which at times could introduce security gaps. Secure coding and architecture standards help these teams understand key do's and don'ts of secure product architecture and development. At the same time, secure coding guidelines will also help to ensure that the entire company codebase follows the same standards. Threat Modeling: Threat modeling helps in identifying the four "Whats" – "What is being built?", "What can go wrong with it?", "What can we do about it?", and "What we finally did." Threat modeling is a way to identify security gaps, ways to fill these gaps, and ways to track if the recommended security standards were followed during system design. In many ways, threat modeling helps an organization quantify risks and vulnerabilities. This results in not only fewer security flaws appearing in live assets, but is also another key factor in helping to prioritize where resources should be spent to reduce overall threat surface.	Secure Coding and Architecture – The goal of defining a secure coding and architecture standard is to minimize exploitable vulnerabilities in products and applications. The following steps can be taken to define these standards: - Identify the tech stack used by your organization and define a secure coding standard by following a guide such as the OWASP Secure coding practice guide. - Depending on whether your environment is set up (cloud, on-prem, etc.), define secure design patterns for developing and deploying an application. The following design principles should be followed - Zero Trust - Least Privilege - Defense in Depth - Secure by default Threat Modeling – Define a threat modeling process which is dependent on a framework suitable for your organization (e.g. STRIDE, DREAD, etc). Because completely manual threat modeling can be time consuming and prone to human error, most organizations opt to use a threat modeling tool, either open source or commercial, which fits their specific needs. All new changes should be threat modeled following this process. All existing implementations should be retroactively threat modeled and any identified risks should be prioritized and minimized within a timeframe acceptable to your organization.	- Request and review current versions of the organization's policies and related documents that are relevant to their software development lifecycle and build and release process. - Ensure that these documents establish secure coding and architecture standards, and threat modeling expectations. - Ensure that one or more of the design principles of Zero Trust, Least Privilege, Defense-in-depth, and Secure-by-default are baked into the SDLC process. - Ensure that some form of threat or risk modeling is integrated to the SDLC process.

Periodically re-evaluate existing threat models and take action on newly identified security risks as they are discovered.

Expected outcome: (1) Your organization has a defined secure coding standard or checklist for each language used for development, a defined security architecture standard that follows the principles of zero trust, least privilege, defense-in-depth, and secure by design, and (2) threat modeling is integrated into your SDLC process.

Security Requirement

Encrypt sensitive data-in-transit:

51. Utilize only strong, trusted methods for encrypting sensitive data in transit
52. Ensure in-transit encryption is used for internal communications as well as public-facing communications
53. For highly-sensitive data, utilize digital signature verification or payload encryption to ensure only trusted destinations can access the data

Encrypt sensitive data-at-rest:

54. Utilize encryption that prevents access to sensitive data while stored on disk
55. For highly sensitive data, ensure encryption occurs at the application level, and does not only rely on volume or disk-only encryption
56. Ensure encryption keys are stored separately from the data they encrypt, are sufficiently diverse, and are rotated appropriately

Requirement Context

Proper encryption ensures the security of sensitive data in storage, and in transit by minimizing exposure.

Encryption in-transit:

The exchange of sensitive information over the web exposes that data to unauthorized external parties. Encrypting the data in transit, minimally by encrypting the communication channels through Transport Layer Security (TLS), reduces this exposure.

Encryption at-rest:

If sensitive data is stored on disk without encryption, any person or service with access to that storage device will also have access to the data. Encrypting the data, prior to it being written to disk, mitigates this risk.

Implementation Guides

Encrypt sensitive data-in-transit – Please see the [Additional Implementation Guidance](#) section for detailed implementation guidance for this requirement.

Encrypt sensitive data-at-rest – Please see the [Additional Implementation Guidance](#) section for detailed implementation guidance for this requirement.

Expected outcome: Your organization (1) uses strong cipher suites for cryptographic operations, (2) encrypts data-in-transits between client and server using TLS versions that are considered “strong”, and (3) encrypts sensitive data-at-rest at volume level, store level, or application level.

Audit Steps

1. Request and review current versions of the organization’s policies and related documents that are relevant to their usage of cryptography.
2. Ensure that these documents establish encryption requirements for different types of sensitive data being processed or stored by the organization.
3. Verify that the documents establish minimum key strengths and cipher suite requirements for encryption.
4. Verify that the documents establish rotation expectations for keys used by encryption operations.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Define data retention and deletion expectations: 57. Establish a clear data retention and deletion policy 58. Prevent the storage of sensitive data (even temporarily) that is not necessary 59. Ensure data retention and deletion is in-compliance with applicable privacy laws and regulations 60. Automate data deletion and operational compliance with data retention and deletion policies 61. Utilize strong data sanitization methods for equipment and media prior to reuse or retirement Tokenize and hash highly sensitive data: 62. Use non-identifiable, non-reversible tokenization or hashing functions to replace highly sensitive data	Reducing the amount of sensitive data that persists in an environment also reduces that environment's overall security risk profile, and the risk of non-compliance with applicable privacy laws. Defined retention and deletion: Sensitive Consumer data that is collected and stored by an organization must be the result of intentional decisions that are aligned with the organization's business objectives, and are in compliance with applicable law. A defined data retention and deletion policy enables an organization to clearly establish data deletion expectations, and an automated data deletion schedule that is in compliance with this policy, enables compliance with applicable privacy laws. Tokenizing highly sensitive data: Replacing highly sensitive data with non-reversible tokens, that still retains their usability for the organization's use case, is an effective way to reduce the overall security risk profile of an environment. A good practical example of this is the use of Plaid's /auth product by developer customers via processor tokens instead of directly handling account and routing numbers.	Defined data retention and deletion – The retention and deletion of sensitive data in your environment, such as consumer data retrieved from the Plaid API, must be intentional. Your organization should establish a data deletion and retention policy that aligns with your business objectives, and is in compliance with applicable law. This policy must be enforced and govern all of your internal data handling practices. Where possible, the enforcement of this policy should be automated. For example, by automating the deletion of sensitive data that no longer serves a business function. Tokenization – Your organization's data handling strategy must consider the use of non-identifiable and non-reversible tokens to minimize the processing and storage of highly sensitive data by individuals and services in your environment. In the context of data retrieved from the Plaid API, we offer tokenized versions of our API endpoints that allow your organization to use the full functionality of the API while only handling tokens. An example of this tokenization is the replacement of account and routing numbers with processor tokens by the Plaid API. For specific implementation guides related to Plaid's /auth product with tokenization, please see our public API documentation @ https://plaid.com/docs/auth/partnerships/ Expected outcome: Your organization has (1) a defined data deletion and retention policy that is in-compliance with applicable law, (2) this policy is communicated and enforced by a collection of technical and organizational controls, and (3) your data handling practices are optimized to replace highly sensitive data with tokens when possible.	1. Request and review current versions of the organization's policies and related documents that are relevant to their data handling, deletion, and retention strategy. 2. Ensure that the strategy sets clear expectations and schedules for deleting obsolete sensitive data. 3. Ensure that the strategy addresses compliance with applicable data privacy laws. 4. Ensure that the strategy utilizes tokenization of highly sensitive data, whenever possible, to achieve data minimization.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Maintain a reliable audit trail: 63. Maintain an audit trail of access and authentication events for critical assets (successful and failed) 64. Maintain an audit trail of all changes to critical assets (successful and failed) Maintain robust alerting: 65. Establish baselines for expected behavior and alert on anomalies or drifts from those baselines	A reliable audit trail, and robust alerting provides an organization with visibility into how assets are behaving. This visibility is critical to ensuring that those assets are secure and available. Audit trails: Monitoring and auditing critical events that occur in an asset provides visibility into how that asset is behaving, and how users and services are interacting with it. Events such as access and authentication attempts, and changes to an asset are especially critical as preventing unauthorized access and changes to an asset are essential for security. Alerting: Appropriately scoped, and sufficiently verbose, logs can produce a lot of log entries, which can make their review challenging. Robust alerts allow an organization to zero-in on events that require attention (e.g. anomalies and drifts of expected behavior), and ignore events that do not (e.g. normal and expected behavior).	Audit trails – Enable or deploy logging for all authentication events and changes to critical assets. These logs must be sufficiently verbose, aggregated into a central repository, protected against unauthorized deletion, and retained for an appropriate period of time. Minimum attributes to capture in logs include (1) date and timestamps, (2) unique user identifiers, (3) environment details (e.g. source and destination of requests), and (4) outcome of events (e.g. success or failure). Logs should be centrally aggregated, and minimally maintained for a year to enable troubleshooting, future audits and investigations. However, the actual retention period beyond a year should align with your organization's unique needs. Alerting – On-call teams and service owners should be alerted in real time of events that require their attention. These alerts should be tuned appropriately to increase accuracy, and reduce false positives. Alerts should have automated escalation paths to ensure that they get triaged in the event that primary on-call personnel are unavailable (e.g. alerts should fire and escalate until they are acknowledged by somebody). Expected outcome: Your organization (1) maintains a robust audit trail for all material events that occur in your environment, and (2) a reliable alerting mechanism that enables real-time detection and triage of events that may negatively impact the security or availability of your environment.	1. Request and review current versions of the organization's policies and related documents that are relevant to their logging, monitoring, and alerting strategy. 2. Ensure that the logging, monitoring, and alerting strategy applies to access and authentication events, as well as changes to critical assets. 3. Ensure that the logging, monitoring, and alerting strategy establishes retention requirements for audit logs. 4. Ensure that audit logs are verbose enough and protected against tampering or deletion. 5. Ensure that critical assets are configured to alert on anomalies and drifts from expected behavior, and on-call resources have been allocated for responding to alerts in real-time.

NIST CSF Mapping: PR.IP-9, PR.IP-10, DE.AE-2, DE.AE-3, DE.AE-4, DE.AE-5, DE.CM-1, DE.CM-2, DE.CM-3, DE.CM-6, DE.CM-7, DE.DP-1, DE.DP-2, DE.DP-3, DE.DP-4, DE.DP-5, RS.RP-1, RS.CO-1, RS.CO-2, RS.CO-3, RS.CO-4, RS.CO-5, RS.AN-1, RS.AN-2, RS.AN-3, RS.AN-4, RS.AN-5, RS.MI-1, RS.MI-2, RS.IM-1, RS.IM-2, RC.RP-1, RC.IM-1, RC.IM-2, RC.CO-1, RC.CO-2, RC.CO-3

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Detect incidents: 66. Implement effective audit logs throughout the platform and corporate resources that support the business 67. Define critical events and enable alerting and reporting that expose these events in real time Respond to incidents: 68. Clearly define a response process to handle both minor and major alerts and incidents 69. Establish and train a response team on how to respond to alerts and incidents 70. Track and categorize incidents and events for later analysis to improve controls and reduce their recurrence over time 71. Perform root cause analysis in order to properly identify and address the sources of incidents	Anticipating and planning for incidents that may impact the security of an environment enables an organization to reduce the potential damage that may be caused by a security incident. Detection: Alerting on-call teams in real time when services and environments deviate from expected behavior is a critical step that enables an organization to appropriately manage their incidents. Response: A defined response procedure that is to be followed by a trained response team when triaging incidents reduces response times, suboptimal decision making, and enables an organization to quickly contain the potential damage that can be caused by a security incident.	<u>Incident detection</u> – The alerts that originate from your environment must be routed to on-call teams in real-time, and those teams must have reliable audit logs for performing investigations. If alerts are too noisy, that will lead to alert-fatigue. Therefore, they must be configured to trigger on material events that indicate that the behavior of an asset is deviating from expected behavior. What is “normal” and what is “abnormal” behavior for an asset should be determined by subject matter experts that have significant depth into that asset. <u>Incident response</u> – The steps that are to be followed by on-call teams when responding to incidents must be defined in an incident response plan or runbook. This incident response plan must establish criteria for identifying the severity of the incident (e.g. severe, high, moderate, low), establish roles and responsibilities, communication channels, and internal escalation paths. The most common incident response roles to consider when building an incident response plan are: - Incident manager or commander – This role has ownership and authority during the incident, and is responsible for coordinating and executing the response. - Communications owner – This role is responsible for coordinating all communication related to the incident to internal and external stakeholders. - Documentation owner – This role is responsible for documenting the timeline of the incident, important decisions, and their outcomes on the incident. - Subject matter experts – These are experts that are called into the incident to provide subject matter expertise and support in responding to the incident. Expected outcome: Your organization has a robust and defined approach for managing an incident from detection through resolution.	- Request and review current versions of the organization's runbooks and related documents that are relevant to incident response. - Ensure that the incident response documents establish criteria for categorizing incidents, minimally, into minor and major incidents. - Ensure that the incident response documents address detection, triage, resolution, and root cause analysis. - Ensure that the incident response documents address notification requirements to external parties that may be impacted by the incident

Security Requirement

Maintain segmented networks:

72. Ensure networks are appropriately segmented based on the assets in each segment (e.g. public vs. filtered vs. private).
73. Ensure segregation between separate networks that host assets of different sensitivities (e.g. production vs. development vs. corporate vs. guest).

Requirement Context

Placing assets with different levels of sensitivity and usage in the same network (e.g. a flat network architecture) leaves highly sensitive assets exposed to unauthorized network traffic.

Segmented networks:

The opposite of a flat network is a segmented network, and it allows an organization to granularly separate assets with different levels of sensitivity and usage from exposure to each other, and to other networks (e.g. the WWW).

An asset with no reason for having access to the internet (e.g. a database containing sensitive data that only interacts with internal services), can live inside a private segmented network, while an asset that have a reason for being visible to the open internet (e.g. a reverse proxy / load balancer) can live inside a network that is open to the web. Both of these assets should not exist in the same subnetwork because their sensitivity, purpose, and network access needs are different.

Implementation Guides

Segmented Networks – Please see the [Additional Implementation Guidance](#) section for detailed implementation guidance for this requirement.

Expected outcome: Your organization's cloud and on-prem networks are segmented based on the sensitivity of assets in those networks, and their needed exposure to the open internet.

Audit Steps

1. Request and review current versions of the organization's policies and related documents that are relevant to their network security strategy.
2. Ensure that these documents address network segmentation of assets based on their sensitivity and connectivity needs to other networks.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Ensure security awareness: 74. Provide awareness training that helps employees recognize abnormal activity on their devices and networks 75. Include recognition of social engineering tactics and defenses in employee training 76. Provide clear reporting and escalation guidelines for employees who identify a security issue	Employees of organizations are a frequent target for phishing and social engineering attacks. Appropriate awareness training is a key control that reduces the likelihood of success of these types of attacks. Security awareness: The techniques used by attackers for phishing and other social engineering evolves over time. If employee awareness does not also evolve, then the organization will stand vulnerable to these attacks. Deliberate and clear awareness campaigns that educate employees on how to spot these attacks, and how to respond when they detect such activity, will reduce their susceptibility to phishing and social social engineering attacks.	<u>Security Awareness</u> – All employees and contractors should be required to take security awareness training before they take on their job responsibilities as new hires. All active employees and contractors should frequently be exposed to security training campaigns and exercises that heighten their awareness. Classroom-style training can lose its effectiveness over time, therefore, implementing high-visibility security training campaigns (e.g. internal phishing exercises) is a necessary supplement to ensure high level of employee engagement, and to measure the security awareness of employees and contractors. Security awareness exercises and training should always include a call to action, and consistent instructions for how to report and escalate suspicious events. Expected outcome: Your organization trains all employees and contractors on security awareness during on-boarding, and continues this training on a recurring basis using up to date training materials. Your organization also has tools for measuring the effectiveness of your employees and contractors.	1. Request and review current versions of the organization's policies and related documents that are relevant to security awareness training. 2. Ensure that these documents capture security awareness training as a requirement for all employees, and outline a method for maintaining security awareness for the organization's workforce. 3. Ensure that the organization has a way to measure the security awareness of its employees and contractors.

Security Requirement

Identify and mitigate third-party vendor risks:

77. Establish and enforce a process for evaluating security risks associated with third-party service providers prior to entering into a commercial relationship, and on an on-going basis after a commercial relationship is established.

Requirement Context

A third-party vendor is another organization that you rely on for performing a critical function on your behalf. Because you do not have complete control or visibility into a third-party vendor, the risks they pose to the security of your organization is anchored to the level of access the vendor has to your environment and data.

The most common type of vendors, in the context of modern technology stacks, are cloud service providers used for hosting applications and infrastructure.

Managing third-party vendor risks:

A vendor intake and monitoring process enables your organization to control how vendors are used to support your business, and to identify and mitigate potential security risks that may arise from that engagement.

The need to identify and mitigate third-party vendor risks are especially crucial for information security when vendors have access to your data or environment..

Implementation Guides

Managing third-party vendor risks – Outsourcing an activity that is critical to your organization must be gated by a defined third-party vendor intake and monitoring process. The intake process must evaluate the risks associated with the service being provided by the vendor, the exchange of information between your organization and the vendor, and any risks to security that may be introduced by the vendor relationship.

This process must establish clear principles around third-party vendor access to your sensitive data and environment, along with criteria that the vendors must meet in order to establish a commercial relationship with your organization. Where possible, this process should also limit vendor access to sensitive data.

This defined process must be clear to all personnel that may engage a third-party vendor, and must be supported by controls that prevent the unauthorized use of third-party vendors that do not follow the vendor intake and monitoring process. These preventive controls can typically be deployed using endpoint security tools (that block unauthorized SaaS services) and expense management processes (blocks fulfillment of invoices from unauthorized services).

Evidence of the vendor intake and monitoring process should be maintained for future records using an internal records keeping or ticketing system (e.g. like Atlassian JIRA).

Expected outcome: Your organization has a defined vendor intake and monitoring process that is communicated to the company, and is enforced by technical and administrative controls.

Audit Steps

1. Request and review current version of the organization's policies and related documents that are relevant to third-party vendor management.
2. Ensure that a gating mechanism has been established to intake and evaluate vendors, and to monitor vendors on an on-going basis.
3. Ensure that clear constraints around vendor access to the organization's environment and data has been established.
4. Ensure that a process exists for evaluating the security risk of a vendor to the organization, in the event that the nature of the engagement introduces the organization to information security risks.

Security Requirement	Requirement Context	Implementation Guides	Audit Steps
Independently stress test and audit controls: 78. Establish a process to enable independent third-parties to stress test the robustness of security defenses, and responsibly disclose their findings to you. 79. Establish a recurring audit cycle for third-party security auditors to audit the design and operational effectiveness of your information security program against internal policies and industry standards.	After building an information security program, and deploying the necessary controls to mitigate against the risks that are relevant to your organization, it may still be difficult to determine how effective those mitigations are. If those mitigations are ineffective, you may not be able to detect this fact on your own before it leads to a security incident. Independent testing: Exposing an organization's products and services, to be independently stress tested by third-party security researchers and auditors, is a good way to identify vulnerabilities before they lead to actual security incidents. Similarly, independently testing the design and operational effectiveness of an organization's information security program is a good way to ensure that the organization's information security objectives are being met.	<u>Independent testing</u> – Independent testing of your organization's security defenses should be performed by third-party security auditors and researchers. Application and network penetration tests should be performed at least annually using a reputable security testing vendor. As external assets (assets exposed to the public internet) are more exposed than internal assets (assets that are in a private internal network), unauthenticated penetration testing of external assets should be prioritized over testing private assets. Authenticated and internal penetration tests should be performed following successful external penetration tests. The implementation of a bug bounty program through a reputable third-party service (e.g. HackerOne) is another good way to implement independent testing of your organization's defenses. The boundaries of the bug bounty program should be well defined to incentivize high quality and continuous independent testing of an organization's defenses, and to enable responsibly disclose findings. At least annually, the design and operational effectiveness of all controls that make up an organization's information security program should be performed by an independent auditor. A third-party assurance program that audits against SSAE18 Trust Service Criteria (SOC 2 Type II) for Security and Confidentiality is a good way to implement third-party auditing of security controls. Expected outcome: Your organization stress tests your security defenses, and subjects your information security program to testing by independent third-party security auditors and researchers.	1. Request latest copies of the organization's third-party audit reports. These can be independent network pen-tests, code scans, bug bounty program summaries, or other security assurance reports like a SOC 2. 2. Ensure that these reports are current (performed within the last 12 months), are performed by reputable independent third-party auditors or pen-testers, and are not internal assessments or scans. 3. In the event that there are material findings, ensure that the organization has the ability to address them effectively.

Cryptography

STRONG CIPHER SUITES

- Acceptably strong algorithms change over time as new techniques for breaking or bypassing their security are discovered. With the rise of ubiquitous cloud computing, historically strong ciphers that were based on compute limitations can now be easily bypassed. To protect against these, ensure cipher suites used to encrypt sensitive data are resistant to both online and offline attacks. Additionally, for symmetric encryption algorithms, distributing the key material is often the determining factor to how strong the encryption being used is. As such, key exchange must be performed out-of-band or otherwise use algorithms that withstand snooping or interception.

ENCRYPTING DATA-IN-TRANSIT:

- For protecting sensitive data while it is in transit, Mozilla maintains the latest configuration samples for many common web and cloud technologies. This can be found at: <https://ssl-config.mozilla.org/>
- The *Intermediate* encryption levels should be considered the absolute minimum for sensitive data-in-transit that needs a broad level of compatibility support across clients and browsers. This limits the server to TLS 1.2 and TLS 1.3, with associated specific cipher restrictions to only still-known strong encryption algorithms.
- For highly sensitive data, the *Modern* settings can be used if all clients accessing the systems support TLS 1.3.
- In cases where compatibility requires that TLS 1.0, TLS 1.1, or weaker ciphers are used (often due to connectivity with legacy systems), strong data-at-rest algorithms should be used to protect the sensitive data prior to transfer. This is often referred to as payload encryption or message-level encryption.

ENCRYPTING DATA-AT-REST:

- Data stored at-rest should be secured using encryption either at the volume level, database level, or application level, depending on the sensitivity of the data.
- For low-sensitivity data, it's recommended to encrypt with at least AES-128 or stronger symmetric encryption.
- For moderately sensitive data or higher AES-256 encryption should be used to provide better robustness against long term attacks and offline decryption attempts.

- Highly sensitive data can be encrypted using Key encapsulation techniques such as envelope encryption, commonly used by tools like Amazon Web Services (AWS) KMS when encrypting data, or using strong asymmetric encryption where it's possible and performance-feasible.
- When implementing encryption for data at rest, multiple layers are often used in order to ensure defense-in-depth, and to reduce overall load and performance impacts of the encryption/decryption operations:
 - Volume-Level Encryption
 - For lower-sensitivity data, simple volume-level encryption, such as using self-encrypting drives, bitlocker, AWS EBS encryption, is an acceptable method of encryption.
 - This is especially useful when protecting workstations and other local storage arrays, or system drives in the cloud.
 - This type of encryption on its own is suitable for internal business data (e.g. such as internal communications, documentation, or other non-public information that is not production-related), or customer data that is not considered sensitive.
 - What's important to remember with volume level encryption is that it's primarily protecting against a single threat vector—an attacker that physically removes the hard drive and attempts to access its data from another system.
 - This means that while the disk is logically accessible to the original server, the data is available to the Operating System (OS) of that server, therefore, anything running on that server.
 - This is why Volume-level encryption should be combined with database-level or application-level encryption when storing moderate to highly-sensitive data.
 - Database-level Encryption
 - For moderately sensitive data, such as the totality of client-provided production data, database-level encryption may be better suited.
 - This is often what is referred to as Transparent Data Encryption (TDE) in SQL server, MongoDB Enterprise's Encrypted Storage Engine, or other database and filestore engine features.
 - Often this is implemented as an additional measure when fully supporting application-level encryption is not an option.

- These are primarily used when trying to ensure that nothing outside of the database engine (or filestore service) is able to access the data, even if they are on the same host.
- This narrows the exposure of the data, but still maintains the risk that the data is exfiltrated through the host database or filestore service and then is now available to the attacker
- Application-Level Encryption
 - Application-level encryption, also known as object-level encryption for key:value data structures, and column-level encryption for relational data structures, is what should be used for highly sensitive data (e.g. regulated PII that must be decryptable, or key data that would be considered a high risk to have exposed).
 - Data suitable for application-level encryption is often a small subset of data that is considered highly valuable to a potential attacker or the company itself.
 - This level of encryption needs to be built into the application at code-level (e.g. using whatever libraries and services are appropriate for the platform).
 - When deployed, this type of encryption will encrypt the highly sensitive plain-text data individually prior to being written to a database..
 - This results in the data only being visible to the application (and to users with authorized access to the application).
 - Additionally, this also results in making bulk data decryption extremely computationally costly for an attacker in the event that an attacker is able to get access to the encrypted data along with the keys.
- There are also other defense-in-depth cryptographic techniques that can be used alongside encryption to reduce or isolate highly sensitive data to either remove the risk of exposure completely or at least drastically increase the difficulty of exfiltration.
- Secure Hashing:
 - For data that needs to be relatively unique, but the original data itself does not need to be maintained, secure hashing mechanisms should be implemented.
 - These are most useful for specific types of data like passwords or generating unique, but non-identifiable fields from PII.
 - Algorithms like argon2, bcrypt, and PBKDF2 are all proven strong algorithms for hashing, but like all hashing algorithms, should be paired with a salt to prevent rainbow-table and other offline attacks.
 - Pairing with a non-stored pepper value makes these even harder to attack, as even if the entire dataset is accessed, the attacker would not have all the data needed to generate their own hashes to compare to the hashed data.
- Tokenization:
 - Tokenization is the process of extracting the sensitive data, and generating a random identifier for that data in its place.
 - Instead of retaining the sensitive data in an encrypted field in the database with associated other data that may result in correlated data causing large amounts of damage, the identifier for the data is stored with the data, and the sensitive data is encrypted at the application level and vaulted away in its own repository.
 - The token vaults often have zero attribution between any of the encrypted data stored, in order to make theft of the vault useless without the other information in the database itself.
 - This is often seen when having to store Payment Card or Personal Health Records (PHI). As an example, a credit card number would be tokenized and vaulted, and the primary database for the application would get an identifier and the expiration date and card holder name. Without breaching both the application database and the vault, credit card fraud can't be performed.

Network Security

SEGMENTED NETWORKS

- Network Segmentation implementations will differ based on the infrastructure in use and the network controls available to the environment. Cloud providers such as Amazon Web Services (AWS) have several services that can be used to provide network segmentation capabilities. On-premise network segmentation will require things like local firewalls and routers (often the same devices) and switches which support functionality like VLANs (which avoid the need for physically separate switches for different network environments). While the below implementation descriptions are examples of network segmentation methods, every company's infrastructure needs are different, and so these examples act to demonstrate how it can be put into practice. In most cases the rules for segmenting networks should be as close to the following as possible:
 - Segmentation should exist between at least production and the development / testing environment using network-level tooling. If a staging environment is in use, that should be separated from both as well. Ex: Production systems should not be able to hit development, and development systems should not be able to affect production.
 - Highly sensitive data should be separated from low-sensitivity or higher risk systems and access into the highly sensitive network should be fully restricted except where explicitly necessary. Ex: Databases should be separated from web services and only necessary ports into those database networks should be allowed.
 - Inbound access to all networks should be limited to only business-necessary services. Ex: A web service may need HTTPS allowed to its servers, whereas workstation environments generally should never need to allow inbound from the internet, but may require allowing access from a management service on the corporate network.
 - Outbound access from network segments should be based on the sensitivity of the data or functions in that segment. Ex: Database environments generally would not need outbound traffic allowed at all, but workstation environments obviously would need to allow web browsing, email, and other similar activities. A service that uses a public API may only need access to that API destination.
 - Segmentation controls should be reviewed regularly to ensure that only access that is still necessary remains in place. Ex: Reviewing security groups, network

ACLs, routing tables, VPC peering, and transit gateways in AWS or firewall rulesets and routing tables in an on-premise environment.

- As with all security measures, segmentation controls should be designed to limit the overall impact of a future breach, not just to protect against internal accidental access

AWS NETWORK SEGMENTATION

- AWS has many services that can be used for implementing network segmentation in the cloud:
 - Segmenting using AWS Accounts:
 - Segmenting production AWS environments from non-production by using different AWS accounts is the strongest method of isolating the environments.
 - While things like transit gateways can provide interconnectivity needed to manage multiple accounts at once from a network layer, separate accounts ensure that teams that need higher-privileges to the cloud account for testing or development do not have the same access in production.
 - This means misconfigurations and mistakes that may occur during development efforts will not directly result in those same events being able to occur on the production account(s).
 - While this is a harder level of segmentation, AWS has provided services over the years to simplify the management of multiple AWS accounts. Therefore, it should be explored as a potential network segmentation solution
 - Segmenting using Virtual Private Clouds (VPCs):
 - AWS's VPC service allows for fully private networks and services to be deployed. Having separate VPCs for production, testing, and staging environments is the common starting point for most segmentation efforts.
 - VPCs incorporate dedicated network subnets, routing tables, gateways, and similar that work together to ensure that communication between two VPCs are intentionally opened, is highly restricted, and can be terminated quickly if necessary.
 - While this protects most of the networking services from cross-segment communication, it does retain the risk that permissions for development may accidentally cause production cloud changes since they are all in the same AWS account.

- Segmenting using Subnets:
 - Inside an AWS VPC, subnets provide the best methods for network segmentation.
 - At a minimum, VPCs that host web-facing services should have at least two subnets (per Availability Zone at the company's discretion). One should be public-facing (which supports Elastic IPs) and the other should be private (no public IP assignments).
 - This ensures that internet-facing systems cannot reach out to the private system except on already authorized ports if they are breached, limiting an attacker's set of tools or techniques they can use to get deeper into the environment.
 - Additionally, consideration should be taken to determine if data storage areas, such as databases, logs, or caching systems, should be separate from both the private and public subnets. Doing so can help reduce overall exposure of those systems, and help contribute to the difficulty an attacker would face trying to get to the most sensitive data in the environment.
- Segmenting using Security Groups:
 - Finally, where possible in AWS, all systems should be protected with very limited Security Group rulesets.
 - If systems don't need outbound, don't allow it. If systems only need HTTPS in, make that the only thing allowed. This helps ensure defense-in-depth by reducing the overall exposure of each individual system and ensures services like SSHD cannot be abused by systems that may have already been compromised.
- A Note about management tools:
 - Management tools will often require the need to access both production and non-production environments, or maintain visibility into highly sensitive environments.
 - It's highly recommended that these tools live in an area outside all production or testing environments (like it's own VPC or account). That way, any SSH access needed, for example, is limited to a subnet that cannot be accessed if a production system was breached.
 - Special care is needed for these management or utility segments, as the assumption is that they will need advanced access.
 - Events like data exfiltration and unusual traffic patterns in networks that house management tools should be monitored as these networks act as management gateways to all environments being controlled.
 - Strong access and authentication controls should be a starting point for management environments, and verbose logging should be used for potential forensic review and alerting on misbehavior of trusted administrators.

Version	Date	Change description
.1	09/18/20	Draft proposed
.2	12/09/20	Implementation guidance updated.
.3	03/02/21	Implementation guidance updated.
1.0	03/24/21	Finalized v1 implementation guides and audit steps.
1.1	06/22/21	Updated name to "Open Finance Data Security Standard"
1.2	1/29/22	Substantive updates that reflect feedback from partners. Control domains are mapped to NIST CSF (where applicable). Application security and SDLC domains are integrated.

Definitions

- **Mutable Servers or infrastructure** – Mutable servers are servers, often long-lived, that support users logging into them via SSH or other management methods in order to allow manual configuration, troubleshooting, or modifications. These systems are associated with a risk of drifting from a hardened configuration over time. Their long-lived nature also means that any breach of that system has the potential to stay active in the environment for an extended period of time, increasing the potential for major compromise.
- **Immutable servers or infrastructure** – Immutable infrastructure refers to servers that are never changed or modified after they're deployed. As a result of their immutability, these types of servers tend to live for short periods of time and are torn down and replaced with new servers when they need to be changed or patched.
- **Sensitive Data** – In the context of this document, data is considered Sensitive Data if it is affected by any of the following requirements:
 - Contractual obligations exist that require the protection of the confidentiality, integrity, or availability of the data processed or stored
 - Regulatory or legal requirements exist that require the protection of the confidentiality, integrity, or availability of the data processed or stored
 - Data contains personally identifiable information, or can be reconstituted into personally identifiable information